

Java Backend Architecture

Interview Series

Chapter 17.5

ConfigMaps & Secrets

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.5 — ConfigMaps & Secrets

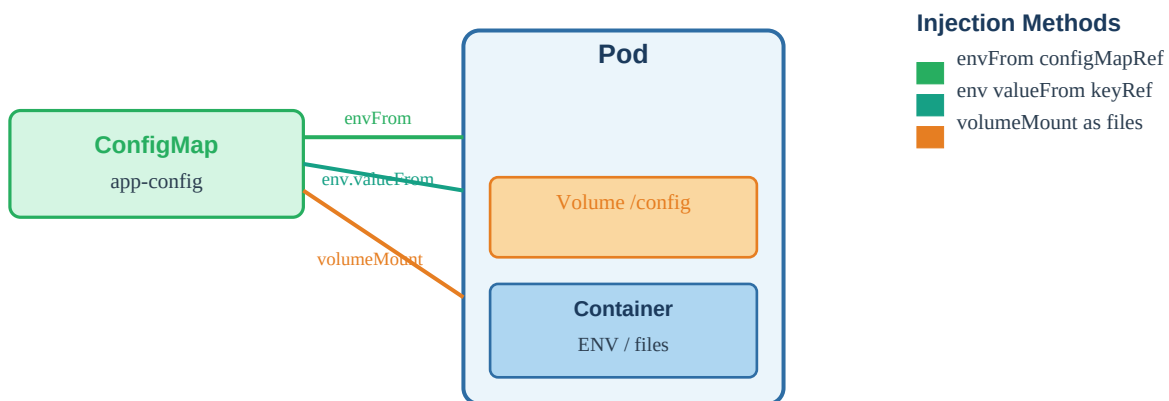
Overview

Kubernetes separates application code from configuration using ConfigMaps (non-sensitive) and Secrets (sensitive, base64-encoded). External Secrets Operator, Sealed Secrets, and Vault Agent Injector extend these primitives for production-grade secret management.

ConfigMap — Non-Sensitive Configuration

Injection Methods:

- **envFrom configMapRef** — load all keys as environment variables into container
- **env valueFrom configMapKeyRef** — inject a specific key as a named env var
- **volumeMount** — mount ConfigMap as files under a directory (live-updated with ~60s delay)



```
# ConfigMap YAML
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  config.properties: |
    db.pool.size=20
    cache.ttl=300
---
# Pod using all three injection methods
spec:
  containers:
    - name: app
      envFrom:
        - configMapRef:
            name: app-config
      # all keys as env vars
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: LOG_LEVEL
      # specific key
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
      # files
      volumes:
        - name: config-volume
          configMap:
            name: app-config
```

Secret — Sensitive Data

- **base64 encoded, NOT encrypted** at rest by default — anyone with etcd access can read
- **Type: Opaque** — arbitrary key/value (most common)
- **Type: kubernetes.io/tls** — tls.crt + tls.key for TLS termination
- **Type: kubernetes.io/dockerconfigjson** — image pull credentials
- **Type: kubernetes.io/service-account-token** — auto-mounted SA tokens
- Same injection methods as ConfigMap: envFrom, env.valueFrom, volumeMount

```
# Create secret imperatively (safe - no base64 in shell history)
kubectl create secret generic db-creds \
  --from-literal=DB_PASSWORD=s3cr3t \
  --from-literal=DB_USER=admin
# Secret YAML
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
data:
  DB_PASSWORD: s3cr3t
  DB_USER: admin
```

```
type: Opaque data: DB_PASSWORD: czNjcjN0 DB_USER: YWRtaW4= # Mount as env var env: - name: DB_PASSWORD valueFrom: secretKeyRef: name: db-creds key: DB_PASSWORD
```

Encryption at Rest — EncryptionConfiguration

By default etcd stores Secrets as base64 plaintext. To enable real encryption, configure EncryptionConfiguration on the API server with an AES-CBC or AES-GCM provider, or use a KMS provider (AWS KMS, GCP CMEK) for envelope encryption.

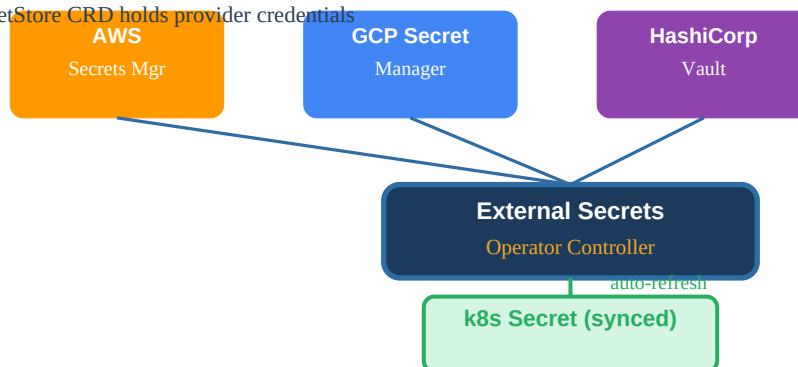
```
# /etc/kubernetes/encryption-config.yaml apiVersion: apiserver.config.k8s.io/v1 kind: EncryptionConfiguration resources: - resources: [secrets] providers: - aescbc: keys: - name: key1 secret: <base64-encoded-32-byte-key> - identity: {} # fallback for existing unencrypted secrets # Start API server with: # --encryption-provider-config=/etc/kubernetes/encryption-config.yaml
```

External Secrets Operator (ESO)

ESO bridges external secret stores (AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault) with Kubernetes Secrets. It auto-syncs and rotates secrets based on refreshInterval.

ExternalSecret CRD defines sync policy

SecretStore CRD holds provider credentials

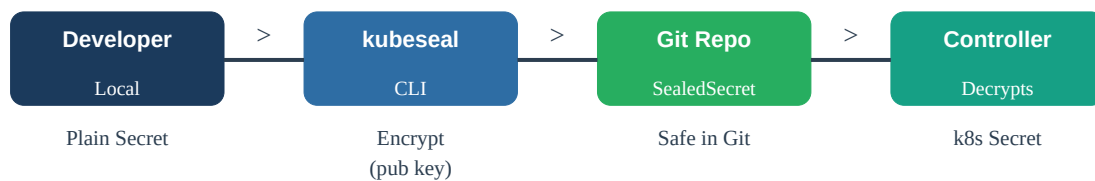


```
# SecretStore pointing to AWS Secrets Manager apiVersion: external-secrets.io/v1beta1 kind: SecretStore metadata: name: aws-store spec: provider: aws: service: SecretsManager region: us-east-1 auth: secretRef: accessKeyID: name: aws-creds key: access-key --- # ExternalSecret syncs to k8s Secret apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: db-secret spec: refreshInterval: 1h secretStoreRef: name: aws-store target: name: db-creds # k8s Secret name data: - secretKey: DB_PASSWORD remoteRef: key: prod/db property: password
```

Sealed Secrets (Bitnami)

Sealed Secrets encrypts a Secret using the cluster's public key (kubeseal CLI). The SealedSecret CRD is safe to commit to Git. The controller decrypts using its private key stored in the cluster.

Sealed Secrets: Encrypt → Git → Decrypt Workflow



Controller holds private key in k8s Secret (cluster-scoped)

```
# Create a SealedSecret from an existing Secret kubectl create secret generic db-creds \
--from-literal=DB_PASSWORD=s3cr3t \ --dry-run=client -o yaml | \ kubeseal
--controller-namespace kube-system \ --format yaml > db-creds-sealed.yaml # Commit
db-creds-sealed.yaml to Git safely # Controller reconciles SealedSecret -> k8s Secret
automatically
```

Vault Agent Injector

- Mutating webhook injects a Vault Agent sidecar into pods with vault.hashicorp.com annotations
- Sidecar authenticates to Vault using Kubernetes Auth method (uses pod SA token)
- Secrets written to shared /vault/secrets/ directory as files — no env var exposure
- Auto-renews lease and rewrites files on rotation

```
# Pod annotation to inject Vault secret annotations: vault.hashicorp.com/agent-inject: "true"
vault.hashicorp.com/role: "app-role" vault.hashicorp.com/agent-inject-secret-db.txt:
"secret/data/db" vault.hashicorp.com/agent-inject-template-db.txt: | {{- with secret
"secret/data/db" -}} DB_PASSWORD={{ .Data.data.password }} {{- end -}} # Secret available at
/vault/secrets/db.txt in app container
```

RBAC for Secrets

- Principle of least privilege: grant get/list only to namespaces/SAs that need it
- Avoid ClusterRole for secrets — use namespace-scoped Role + RoleBinding
- Audit logs: enable secret access auditing in API server audit policy

```
# Role granting read-only access to specific secret apiVersion: rbac.authorization.k8s.io/v1
kind: Role metadata: name: secret-reader namespace: app-ns rules: - apiGroups: [""] resources:
["secrets"] resourceNames: ["db-creds"] # limit to specific secret verbs: ["get"] --- kind:
RoleBinding apiVersion: rbac.authorization.k8s.io/v1 metadata: name: app-secret-binding
namespace: app-ns subjects: - kind: ServiceAccount name: app-sa roleRef: kind: Role name:
secret-reader apiGroup: rbac.authorization.k8s.io
```

Comparison Table

Approach	Encrypted at Rest	Git Safe	Auto-Rotate	Best For
k8s Secret (plain)	No (base64)	No	No	Dev/test only
EncryptionConfig + KMS	Yes	No	No	Production baseline
External Secrets Op.	Yes (in vault)	No secret in Git	Yes	Cloud-native teams
Sealed Secrets	Encrypted in Git	Yes	Manual	GitOps workflows

Vault Agent Injector	Yes (Vault)	No	Yes (lease)	Zero-trust secrets
----------------------	-------------	----	-------------	--------------------

Interview Q&A; — ConfigMaps & Secrets

Q1. What is the difference between a ConfigMap and a Secret?

ConfigMap stores non-sensitive configuration (plain text key-value, properties files). Secret stores sensitive data encoded in base64. Both can be injected via `envFrom`, `env.valueFrom`, or `volumeMount`. Critical difference: Secrets should have RBAC restrictions and encryption at rest enabled; ConfigMaps do not. Base64 is NOT encryption — it is just encoding.

Q2. Why is base64 not encryption?

base64 is a reversible encoding scheme with no key. Anyone who can read the Secret YAML or access `etcd` can run `'echo czNjcjN0 | base64 -d'` and retrieve the plaintext immediately. Encryption requires a key and cryptographic algorithm — e.g., AES-GCM. Enable `EncryptionConfiguration` or use a KMS provider for real protection.

Q3. What are the Secret types in Kubernetes?

Opaque (generic key-value), `kubernetes.io/tls` (tls.crt + tls.key), `kubernetes.io/dockerconfigjson` (registry credentials), `kubernetes.io/basic-auth`, `kubernetes.io/ssh-auth`, `kubernetes.io/service-account-token` (auto-mounted SA token), `bootstrap.kubernetes.io/token`. Opaque is most common for application secrets.

Q4. How does envFrom differ from env.valueFrom?

`envFrom`: `configMapRef/secretRef` loads ALL keys from the ConfigMap/Secret as environment variables. `env.valueFrom.configMapKeyRef/secretKeyRef` selects ONE specific key and maps it to a specific env var name. Use `envFrom` for bulk loading, `valueFrom` when you need to rename or selectively inject.

Q5. What happens to env vars vs volume-mounted ConfigMaps on update?

Volume-mounted ConfigMaps are live-updated by kubelet within ~60 seconds (subject to kubelet sync period). Environment variables injected at pod start are STATIC — pod must be restarted to pick up changes. This is a critical operational difference: for hot config reload, use volume mounts.

Q6. How does External Secrets Operator work?

ESO installs CRDs: `SecretStore` (provider credentials/endpoint) and `ExternalSecret` (what to sync). The controller polls the external store (AWS SM, GCP SM, Vault) at `refreshInterval`, fetches the secret value, and upserts a Kubernetes Secret. If the external value rotates, the k8s Secret is automatically updated. Pod environment vars still need a restart, but volume-mounted secrets update live.

Q7. What is the ClusterSecretStore vs SecretStore?

`SecretStore` is namespace-scoped — only `ExternalSecrets` in the same namespace can reference it. `ClusterSecretStore` is cluster-wide — any namespace can reference it. Use `ClusterSecretStore` for shared infrastructure secrets (database passwords, API keys) and `SecretStore` for tenant isolation.

Q8. How do Sealed Secrets work?

The Sealed Secrets controller generates an asymmetric key pair (RSA-OAEP + AES-GCM). `kubeseal` CLI fetches the public cert from the controller, encrypts the Secret, and outputs a `SealedSecret` YAML. The ciphertext is scoped to a namespace+name combination to prevent secret theft across namespaces. Only the controller can decrypt using its private key.

Q9. How do you rotate the Sealed Secrets key?

The controller generates new keys periodically (default 30 days) but keeps old keys to decrypt existing `SealedSecrets`. You must re-seal all secrets with the new key. In emergency (key compromise), delete the controller's key Secret, restart the controller — all existing `SealedSecrets` become un-decryptable until re-sealed. Back up the controller's key Secret regularly.

Q10. How does Vault Agent Injector authenticate?

The sidecar uses the Kubernetes Auth method. It sends the pod's service account JWT to Vault's `/auth/kubernetes/login` endpoint. Vault validates the JWT against the Kubernetes API server (using `TokenReview`), checks the `bound_service_account_names` and `bound_service_account_namespaces` in the Vault role, and returns a Vault token scoped to the configured policies.

Q11. What is EncryptionConfiguration and how do you enable it?

`EncryptionConfiguration` is an API server config file specifying which resources to encrypt and with which provider. Providers: `identity` (no-op), `aescbc`, `aesgcm`, `secretbox`, `kms` (envelope encryption). Pass `--encryption-provider-config` to `kube-apiserver`. Existing secrets must be re-written: `kubectl get secrets -A -o json | kubectl replace -f -` to encrypt in place.

Q12. What is envelope encryption for Secrets?

The KMS provider generates a DEK (Data Encryption Key) per secret using a random key, encrypts the DEK with a KEK (Key Encryption Key) stored in the KMS (AWS KMS, GCP KMS), and stores the encrypted DEK alongside the encrypted data in etcd. Only the KMS can decrypt the KEK, providing hardware-level key protection and audit trails.

Q13. How do you prevent secret sprawl in a large cluster?

1) Use External Secrets Operator — single source of truth in vault. 2) Enable Secret auditing in audit policy to detect over-permissioned access. 3) Use OPA/Gatekeeper to require secrets be from approved sources. 4) Scan Git history for accidentally committed secrets (GitLeaks, TruffleHog). 5) Use namespace-scoped RBAC to isolate team secret access.

Q14. How does RBAC restrict secret access?

By default, any pod in a namespace with a ServiceAccount can potentially access secrets if RBAC allows. Best practice: create Role (namespace-scoped) with verbs `[get]` and `resourceNames` listing only required secrets. Bind to application ServiceAccount. Avoid ClusterRole granting secrets access cluster-wide. Enable audit logging for secret `get/list/watch`.

Q15. What is the automountServiceAccountToken and why disable it?

Kubernetes auto-mounts the pod's SA token at `/var/run/secrets/kubernetes.io/serviceaccount/token`. If an attacker compromises the pod, they can use this token to query the Kubernetes API. Set `automountServiceAccountToken: false` on pods/SAs that don't need API access. For pods needing API access, bind minimal RBAC permissions.

Q16. How do you handle secret rotation without downtime?

1) Volume-mounted secrets auto-update within kubelet sync period (~60s). Application must watch file for changes and reload. 2) For env vars, use rolling restart (`kubectl rollout restart`). 3) ESO handles external rotation automatically — set `refreshInterval` to match vault TTL. 4) Implement application-level secret refresh (Spring `@RefreshScope + /actuator/refresh`). 5) Use readiness probes to ensure new secrets are valid before routing traffic.

Q17. What is the difference between Secret and ConfigMap volume update behavior?

Both are updated by kubelet within the kubelet's `configMapAndSecretChangeDetectionStrategy` period. With the default `Watch` strategy, updates propagate in ~60s. With `Cache` strategy, it depends on TTL. Note: `subPath` volumeMounts do NOT receive live updates — the file is a static copy at pod start.

Q18. How do you use imagePullSecrets?

Create a dockerconfigjson Secret: 'kubectl create secret docker-registry regcred --docker-server=registry.example.com --docker-username=user --docker-password=pass'. Reference in pod spec: spec.imagePullSecrets: [{name: regcred}]. Or attach to ServiceAccount so all pods using that SA automatically get the pull secret.

Q19. What is the risk of printing secrets in pod logs?

Logs are stored on the node and accessible via kubectl logs. If env vars from Secrets are logged by the application (common in debug mode), they are exposed to anyone with logs access. Mitigation: use volume mounts instead of env vars (harder to accidentally log), implement log scrubbing, restrict kubectl logs RBAC to ops team only.

Q20. How does the Vault Agent sidecar differ from the Vault CSI driver?

Vault Agent Injector: mutating webhook injects init+sidecar containers, writes secrets to shared emptyDir volume, manages lease renewal. Vault CSI Provider: uses secrets-store-csi-driver, mounts secrets as a volume (ephemeral inline volume), can also sync to k8s Secrets. CSI approach is more cloud-native, doesn't require extra containers, but has less flexibility for templating compared to Vault Agent's Consul template.

Q21. How do you audit which pods access which Secrets?

1) Kubernetes audit log: enable audit policy with level=Request for secrets resources. 2) kubectl auth can-i --as=system:serviceaccount:ns:sa get secret -n ns. 3) OPA/Gatekeeper audit: check RoleBindings granting secret access. 4) Falco runtime security: detect unexpected secret access at process level. 5) Tools like Kubiscan or rbac-tool to analyze effective permissions across the cluster.

Q22. What is the recommended approach for a GitOps workflow with secrets?

Use Sealed Secrets or External Secrets Operator. With Sealed Secrets: store SealedSecret YAML in Git, ArgoCD/Flux syncs to cluster, controller decrypts. With ESO: store ExternalSecret CRD (referencing vault path) in Git, controller fetches actual secret from vault. Never store plaintext Secrets in Git — use pre-commit hooks (detect-secrets, gitleaks) to prevent accidents.

Q23. What happens if you delete a Secret that a running pod uses?

For env var injection: pod continues running with the old values in memory — env vars are copied at container start and are not live-linked. For volume mounts: the volume continues serving the last cached content. New pods cannot start if the Secret is missing and the volume mount is required — they will fail with a CreateContainerConfigError.

Q24. How do you store TLS certificates as Secrets?

kubectl create secret tls my-tls-secret --cert=tls.crt --key=tls.key Creates a kubernetes.io/tls Secret with tls.crt and tls.key keys. Reference in Ingress: spec.tls[].secretName. Cert-manager automates creation and renewal, storing Let's Encrypt/ACME certs as kubernetes.io/tls Secrets, watched by Ingress controllers.

Q25. What is the difference between Opaque and other Secret types?

Opaque: no validation, arbitrary key-value. kubernetes.io/tls: must have tls.crt and tls.key. kubernetes.io/dockerconfigjson: must have .dockerconfigjson key with valid JSON. kubernetes.io/basic-auth: must have username and password. Type enforcement helps tools (Ingress controllers, kubelet) understand the secret's purpose and validate format before use.

Q26. How would you migrate from hardcoded env vars to External Secrets Operator?

1) Store current secret values in AWS Secrets Manager / Vault. 2) Deploy ESO controller and create SecretStore with IAM/auth credentials. 3) Create ExternalSecret CRD referencing the vault path. 4) Verify k8s Secret is created and contains correct values. 5) Update pod spec to reference the new Secret (same as before — just different origin). 6) Rolling restart pods to pick up new secrets. 7) Delete old manually-managed

Secrets. Zero-downtime migration.

Q26. How does reloader (Stakater) help with ConfigMap/Secret changes?

Stakater Reloader watches ConfigMaps and Secrets for changes and automatically triggers rolling restarts of Deployments/StatefulSets that reference them. Annotate the deployment with `reloader.stakater.com/auto: 'true'`. This solves the `env var static-copy` problem without manual `'kubectl rollout restart'`, enabling fully automated config update pipelines.

Q27. What is a projected volume and when would you use it?

A projected volume allows multiple volume sources (Secret, ConfigMap, ServiceAccountToken, DownwardAPI) to be mapped into a single directory. Useful for applications that expect all config in one path. Example: project DB cert, app config, and SA token into `/etc/app-config/`. Also used to create short-lived service account tokens with custom audiences via `tokenExpirationSeconds`.

Q28. How do you create a Secret from a file?

`kubectl create secret generic tls-ca --from-file=ca.crt=./ca.crt` The key will be `'ca.crt'` and value will be the file contents. For multiple files: `--from-file=file1 --from-file=key2=file2`. For env file format: `--from-env-file=.env` (each `KEY=VALUE` line becomes a secret key). Avoid `--from-literal` in shell for sensitive values (shell history).

Q29. What is the maxSecretSize limit and why does it matter?

By default, Secrets (and ConfigMaps) have a 1MB size limit enforced by etcd. This prevents large binary blobs from overloading etcd. For large configs, consider external config stores (S3, GCS) with pre-signed URLs, or mount a PVC. The 1MB limit includes all metadata — in practice, keep Secrets under 512KB.

Q30. How does Spring Boot integrate with Kubernetes Secrets?

Spring Cloud Kubernetes reads Secrets as `PropertySources`. Enable with `spring.cloud.kubernetes.secrets.enabled=true` and RBAC granting `get/list` on secrets. Alternatively, use Spring Vault for Vault integration. Best practice: mount Secret as environment variables or files and let Spring's standard `${DB_PASSWORD}` expression resolution handle it — no k8s-specific library needed.

Q31. What is the RBAC impact of using list vs get on Secrets?

`get`: allows fetching a specific Secret by name — requires `resourceNames` restriction. `list`: returns ALL Secrets in a namespace — very dangerous, should be avoided for app SAs. `watch`: streams all Secret events — even more dangerous. Principle: grant `resourceNames-restricted get` only. Never grant `list/watch` to application SAs. Admin and monitoring tools may need `list` but should be audited.

Q32. How do you test that secrets are correctly injected in CI?

1) Use `kubectl apply --dry-run=server` to validate Secret references exist. 2) Deploy to a test namespace with mock Secrets matching expected keys. 3) Use `helm test` or integration test pods to verify env vars/files are present. 4) tools like `Conftest/OPA` can enforce secret naming conventions in CI. 5) ESO has a built-in validate endpoint to check `ExternalSecret` status.

Q33. What is secret masking in CI/CD pipelines?

CI systems (GitHub Actions, GitLab CI, Jenkins) automatically mask secret variable values in log output. Kubernetes does not mask secrets in pod logs — application responsibility. Use structured logging with field-level redaction. Tools like Logstash filters or Fluentd `redact_sensitive` plugin can scrub known patterns (passwords, tokens) from log streams.

Q34. How does cert-manager store and rotate TLS secrets?

`cert-manager` issues Certificates CRD backed by Issuers (Let's Encrypt, Vault PKI, self-signed). It stores issued certs as `kubernetes.io/tls` Secrets. It monitors cert expiry and renews (default: 2/3 of lifetime elapsed). On renewal, it updates the Secret in-place. Ingress controllers that watch the Secret serve the new cert

without restart.

Q35. What monitoring should you add for secret management?

1) Alert on ESO ExternalSecret sync failures (condition Ready=False). 2) Alert on Secret creation/deletion audit events from unexpected service accounts. 3) Monitor cert-manager Certificate expiry (cert_manager_certificate_expiration_timestamp_seconds metric). 4) Falco rules for unexpected secret file reads. 5) Alert when Sealed Secrets controller key is approaching rotation deadline. 6) Vault token TTL and lease renewal metrics.

Q36. What is the difference between stringData and data in a Secret?

data: requires base64-encoded values — must encode manually. stringData: accepts plaintext strings — Kubernetes base64-encodes them on creation. stringData is a write-only convenience field — it does not appear in GET responses. In GitOps where you're generating Secrets programmatically, use stringData to avoid base64 encoding in scripts. Note: never commit plaintext Secret YAML to Git.

Q37. How do you handle multi-tenant secret isolation?

1) Separate namespaces per tenant with namespace-scoped Secrets. 2) NetworkPolicies do not protect secrets — RBAC does. 3) Use SecretStore (namespace-scoped) per tenant in ESO. 4) Vault namespaces/paths per tenant. 5) OPA/Kyverno policies to enforce that pods in tenant-A namespace cannot reference Secrets from tenant-B namespace (cross-namespace references are not native k8s feature but volume mounts and secretKeyRef only work within same namespace anyway).

Q38. How do you back up and restore Secrets?

Velero backs up Secrets as part of namespace backup. However, backed-up Secrets are stored as base64 in the backup storage — ensure the backup bucket is encrypted and access-controlled. For Sealed Secrets: back up the controller's key Secret ('kubectl get secret -n kube-system -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml'). Without it, SealedSecrets cannot be decrypted.